第18章 从猫爷借钱说起——事务简介

18.1 事务的起源

对于大部分程序员来说，他们的任务就是把现实世界的业务场景映射到数据库世界中。比如，银行会为了存储人们的账户信息而建立一个 account 表：

```
CREATE TABLE account (
    id INT NOT NULL AUTO_INCREMENT COMMENT '自增id',
    name VARCHAR(100) COMMENT '客户名称',
    balance INT COMMENT '余额',
    PRIMARY KEY (id)
) Engine=InnoDB CHARSET=utf8;
```

狗哥和猫爷是一对好朋友。他们到银行各自开设了一个账户，这样一来，俩人在现实世界中拥有的资产就会体现在数据库世界的 account 表中。比如，现在狗哥有 11 元，猫爷只有 2 元，那么现实中的这个情况映射到数据库的 account 表就是下面这样：

```
+----+--------+---------+
| id | name   | balance |
+----+--------+---------+
|  1 | 狗哥   |      11 |
|  2 | 猫爷   |       2 |
+----+--------+---------+
```

在某个特定的时刻，狗哥、猫爷这些家伙在银行所拥有的资产是一个特定的值。这些特定的值也可以被描述为账户在这个特定的时刻在现实世界中的一个状态。随着时间的流逝，狗哥和猫爷可能陆续向银行账户中存钱、取钱或者向别人转账，他们账户中的余额也因此发生变动，每一个操作都相当于现实世界中账户的一次状态转换。

数据库世界作为现实世界的一个映射，自然也要进行相应的变动。不变不知道，一变吓一跳。现实世界中一些看似简单的状态转换，映射到数据库世界却不是那么容易。

比如，有一次猫爷着急用钱，急忙打电话给狗哥借 10 块钱。现实世界中的狗哥走向银行的 ATM 机，输入了猫爷的账号以及 10 元的转账金额，然后按下确认键之后就拔卡走人了。对于数据库世界来说，这相当于执行了下面这两条语句：

```
UPDATE account SET balance = balance - 10 WHERE id = 1;
UPDATE account SET balance = balance + 10 WHERE id = 2;
```

但是这里面有个问题。如果上述两条语句只执行了一条时，服务器忽然断电了，这该咋

办？把狗哥的钱扣了，但是没给猫爷转过去，那猫爷还是逃脱不了着急用钱的窘境。即使对于单独的一条语句，上一章在唠叨 Buffer Pool 时也说过，在对某个页面进行读写访问时，都会先把这个页面加载到 Buffer Pool 中；之后如果修改了某个页面，也不会立即把修改刷新到磁盘，而只是把这个修改后的页面添加到 Buffer Pool 的 flush 链表中，在之后的某个时间点才会刷新到磁盘。如果在将修改过的页刷新到磁盘之前系统崩溃了，猫爷岂不是依然陷在用钱窘境中？

怎么才能让可怜的猫爷摆脱窘境呢？其实再仔细想想，我们只是想让某些数据库操作符合现实世界中状态转换的规则而已。设计数据库的大叔仔细盘算了盘算，现实世界中状态转换的规则有好几条，我们慢慢道来。

18.1.1 原子性（Atomicity）

在现实世界中，转账操作是一个不可分割的操作。也就是说，要么压根儿就没转，要么转账成功；不能存在中间的状态，也就是转了一半的这种情况。设计数据库的大叔把这种“要么全做，要么全不做”的规则称为原子性。但是，现实世界中一个不可分割的操作却可能对应着数据库世界中若干条不同的操作，数据库中的一条操作也可能被分解成若干个步骤（比如先修改缓冲页，之后再刷新到磁盘等）。最要命的是，在任何一个可能的时间点都可能发生意想不到的错误（可能是数据库本身的错误，也可能是操作系统错误，甚至还可能是直接断电之类的意外）而使操作执行不下去。为了保证数据库世界中某些操作的原子性，设计数据库的大叔需要花费一些心思来保证：如果在执行操作的过程中发生了错误，就把已经执行的操作恢复成没执行之前的样子。这也是后面章节将要仔细唠叨的内容。

18.1.2 隔离性（Isolation）

在现实世界中，两次状态转换应该是互不影响的。比如，狗哥向猫爷同时进行了两次金额为 5 元的转账（假设可以在两个 ATM 机上同时操作）。那么最后狗哥的账户里肯定会少 10 元，而猫爷的账户里肯定多了 10 元。但是到对应的数据库世界中，事情又变得复杂了一些。为了简化问题，我们粗略地假设狗哥向猫爷转账 5 元的过程是由下面这几个步骤组成的。

1. 读取狗哥账户的余额到变量 A 中；简写为 read(A)。
2. 将狗哥账户的余额减去转账金额；简写为 A = A − 5。
3. 将狗哥账户修改过的余额写到磁盘中；简写为 write(A)。
4. 读取猫爷账户的余额到变量 B；简写为 read(B)。
5. 将猫爷账户的余额加上转账金额；简写为 B = B + 5。
6. 将猫爷账户修改过的余额写到磁盘中；简写为 write(B)。

我们将狗哥向猫爷同时进行的两次转账操作分别称为 T1 和 T2。在现实世界中 T1 和 T2 应该是没有关系的，可以先执行完 T1，再执行 T2；或者先执行完 T2，再执行 T1。对应的数据库操作如图 18-1 所示。

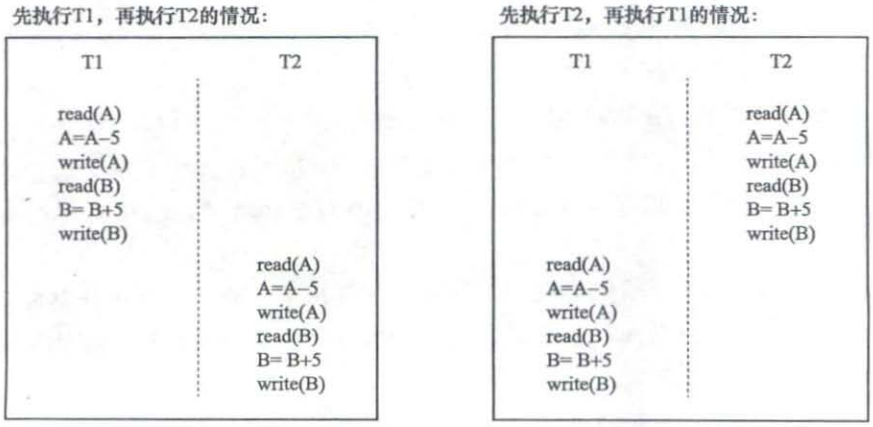

图 18-1　转账操作对应的数据库操作

但是很不幸，在真实的数据库中，T1 和 T2 的操作可能交替执行，如图 18-2 所示。

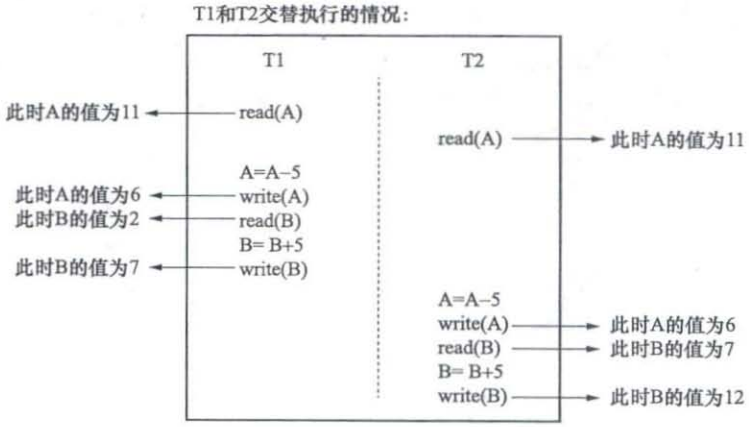

图 18-2　真实的数据库中 T1 和 T2 的操作

如果按照图 18-2 中的执行顺序来进行两次转账，最终狗哥的账户里还剩 6 元钱，相当于只扣了 5 元钱，但是猫爷的账户里却成了 12 元钱，相当于多了 10 元钱。这样一来，银行岂不是要亏死了？

所以，对于现实世界中状态转换对应的某些数据库操作来说，不仅要保证这些操作以原子性的方式执行完成，而且要保证其他的状态转换不会影响到本次状态转换，这个规则称为隔离性。这时，设计数据库的大叔就需要采取一些措施，让访问相同数据（上例中的 A 账户和 B 账户）的不同状态转换（上例中的 T1 和 T2）对应的数据库操作的执行顺序有一定规律，这也是后面章节要仔细唠叨的内容。

18.1.3　一致性（Consistency）

我们生活的现实世界中存在形形色色的约束，比如身份证号不能重复、性别只能是男或者女、高考的分数只能在 0 ～ 750 之间（国内某些省份）、人民币的最大面值只能是 100（现在是 2020 年）、红绿灯只有 3 种颜色、房价不能为负的、学生要听老师话；等等等等。只有符

合这些约束的数据才是有效的。比如，有个小孩儿跟你说他的高考成绩是 1000 分，你一听就知道他在胡扯。

数据库世界只是现实世界的一个映射，现实世界中存在的约束当然也要在数据库世界中有所体现。如果数据库中的数据全部符合现实世界中的约束，我们就说这些数据就是一致的，或者说符合一致性的。

如何保证数据库中数据的一致性呢（就是符合所有现实世界的约束）？这其实是靠两方面的努力。

- 数据库本身能为我们解决一部分一致性需求（就是数据库自身可以保证现实世界的一部分约束永远有效）。

我们知道，MySQL 数据库可以为表建立主键、唯一索引、外键，还可以声明某个列为 NOT NULL 来拒绝 NULL 值的插入。比如，当对某个列建立唯一索引时，如果插入某条记录时发现该列的值重复了，MySQL 就会报错并且拒绝插入。除了这些已经非常熟悉的用来保证一致性的功能，MySQL 还支持使用 CHECK 语法来自定义约束。比如下面这样：

```
CREATE TABLE account (
    id INT NOT NULL AUTO_INCREMENT COMMENT '自增id',
    name VARCHAR(100) COMMENT '客户名称',
    balance INT COMMENT '余额',
    PRIMARY KEY (id),
    CHECK (balance >= 0)
);
```

在这个例子中，CHECK 语句的本意是想规定 balance 列不能存储小于 0 的数字，对应在现实世界中的意思就是银行账户余额不能小于 0。但是很遗憾，MySQL 仅仅支持 CHECK 语法，但实际上并没有用。也就是说，即使使用上述带有 CHECK 子句的建表语句来创建 account 表，在后续插入或更新记录时，MySQL 也不会去检查 CHECK 子句中的约束是否成立。

小贴士

其他的一些数据库（比如 SQL Server 或者 Oracle）支持的 CHECK 语法是有实实在在的作用的，每次进行插入或更新记录之前都会检查数据是否符合 CHECK 子句中指定的约束条件，如果不符合就会拒绝插入或更新。

虽然 CHECK 子句对一致性检查没什么用，但我们还是可以通过定义触发器的方式来自定义一些约束条件，以保证数据库中数据的一致性。

小贴士

触发器是 MySQL 的基础知识，而本书的定位是 MySQL 进阶。如果大家不了解触发器，恐怕就要找本基础的 MySQL 图书来看看了，比方说《MySQL 是怎样使用的：从零开始学习 MySQL》。

- 更多的一致性需求需要靠写业务代码的程序员自己保证。

为了建立现实世界和数据库世界的对应关系，理论上应该把现实世界中的所有约束都反映到数据库世界中。但是很不幸，在更改数据库数据时进行一致性检查是一个耗费性能的工作。比如，我们为 account 表建立了一个触发器，每当插入或者更新记录时都会校验 balance 列的值是否大于 0，这就会影响到插入或更新的速度。仅仅校验一行记录是否符合一致性的需求倒

也不是什么大问题，但是有的一致性需求简直“变态”。比如，银行会建立一张代表账单的表，里面记录了每个账户的每笔交易，而且每一笔交易完成后，都需要保证整个系统的余额等于所有账户的收入减去所有账户的支出。如果在数据库层面实现这个一致性需求的话，则每次发生交易时，都需要将所有的收入加起来，然后再减去所有的支出；再将所有的账户余额加起来，看看两个值是否相等。如果账单表中有几亿条记录，光是这个校验的过程可能就要耗费好几个小时。也就是说你在煎饼摊买煎饼时，使用银行卡付款之后要等好几个小时才能提示付款成功。这不是搞笑么！这样的性能代价是完全承受不起的。

现实生活中复杂的一致性需求比比皆是，而由于性能问题把一致性需求交给数据库来解决也是不现实的，所以这个“锅”就甩给了业务端的程序员。比如我们的 account 表，我们也可以不建立触发器，只要编写业务代码的程序员在自己的代码中判断一下，当某个操作会将 balance 列的值更新为小于 0 的值时，不执行该操作。这就好了嘛！

前文唠叨的原子性和隔离性都会对一致性产生影响。比如，在现实世界中转账操作完成后，有这样一个一致性需求：参与转账的账户的总余额是不变的。如果数据库不遵循原子性要求，比如转了一半就不转了，也就是说给狗哥扣了钱而没给猫爷转过去，那就是不符合一致性需求的。类似地，如果数据库不遵循隔离性要求，就像前面唠叨隔离性时举的例子那样，最终狗哥账户中扣的钱和猫爷账户中涨的钱可能就不一样了，也就是说不符合一致性需求了。所以说，数据库某些操作的原子性和隔离性都是保证一致性的一种手段，在操作执行完成后保证符合所有既定的约束则是一种结果。那么，满足原子性和隔离性的操作一定就满足一致性么？这倒也不一定。比如，狗哥要转账 20 元给猫爷，虽然这满足原子性和隔离性，但是在转账完成后狗哥账户的余额就成负的了，这显然是不满足一致性的。那么，不满足原子性和隔离性的操作就一定不满足一致性么？也不一定，只要最后的结果符合所有现实世界中的约束，那么就是符合一致性的（当然，我们一般在定义一致性需求时，只要某些数据库操作满足原子性和隔离性规则，那么这些操作执行后的结果就会满足一致性需求）。

18.1.4 持久性（Durability）

当现实世界中的一个状态转换完成后，这个转换的结果将永久保留，这个规则被设计数据库的大叔称为持久性。比如，狗哥向猫爷转账，ATM 机提示转账成功时，就意味着这次账户的状态转换完成了，狗哥就可以拔卡走人了。如果狗哥走人之后，银行又把这次转账操作给撤销掉，恢复到没转账之前的样子，猫爷就惨了，所以这个持久性是非常重要的。

当把现实世界中的状态转换映射到数据库世界时，持久性意味着该次转换对应的数据库操作所修改的数据都应该在磁盘中保留下来，无论之后发生了什么事故，本次转换造成的影响都不应该丢失（要不然猫爷就麻烦大了）。

18.2 事务的概念

为了方便大家记住前文唠叨的现实世界状态转换过程中需要遵守的 4 个特性，我们把原子性（Atomicity）、隔离性（Isolation）、一致性（Consistency）和持久性（Durability）这 4 个单词对应的首字母提取出来，就是 A、I、C、D。这 4 个字母稍微变换一下顺序可以组成一个完

整的英文单词：ACID（在英文中为“酸”的意思）。以后我们提到 ACID 这个词儿，大家就应该想到原子性、一致性、隔离性、持久性这几个规则。

另外，设计数据库的大叔为了方便起见，把需要保证原子性、隔离性、一致性和持久性的一个或多个数据库操作称为事务（transaction）。

我们现在知道，事务是一个抽象的概念，它其实对应着一个或多个数据库操作。设计数据库的大叔根据这些操作所执行的不同阶段把事务大致划分成了下面几个状态。

- 活动的（active）：事务对应的数据库操作正在执行过程中时，我们就说该事务处于活动的状态。
- 部分提交的（partially committed）：当事务中的最后一个操作执行完成，但由于操作都在内存中执行，所造成的影响并没有刷新到磁盘时，我们就说该事务处于部分提交的状态。
- 失败的（failed）：当事务处于活动的状态或者部分提交的状态时，可能遇到了某些错误（数据库自身的错误、操作系统错误或者直接断电等）而无法继续执行，或者人为停止了当前事务的执行，我们就说该事务处于失败的状态。
- 中止的（aborted）：如果事务执行了半截而变为失败的状态，比如前面唠叨的狗哥向猫爷转账的事务，当狗哥账户的钱被扣除，但是猫爷账户的钱没有增加时遇到了错误，从而导致当前事务处在了失败的状态，那么就需要把已经修改的狗哥账户余额调整为未转账之前的金额。换句话说，就是要撤销失败事务对当前数据库造成的影响。这个撤销的过程用书面一点的话描述就是：回滚。当回滚操作执行完毕后，也就是数据库恢复到了执行事务之前的状态，我们就说该事务处于中止的状态。
- 提交的（committed）：当一个处于部分提交的状态的事务将修改过的数据都刷新到磁盘中之后，我们就可以说该事务处于提交的状态。

随着事务对应的数据库操作执行到不同的阶段，事务的状态也在不断变化。一个基本的状态转换图如图 18-3 所示。

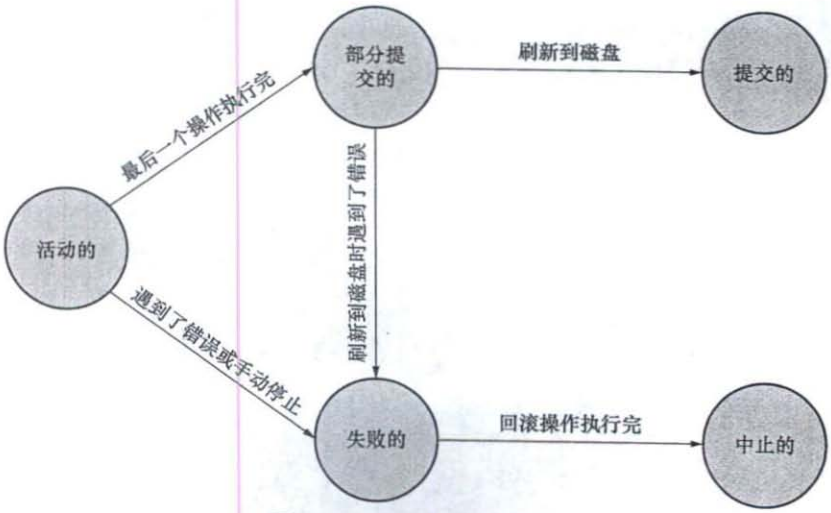

图 18-3 事务的状态转换图

从图 18-3 可以看出，只有当事务处于提交的或者中止的状态时，一个事务的生命周期才

算是结束了。对于已经提交的事务来说，该事务对数据库所做的修改将永久生效；对于处于中止状态的事务来说，该事务对数据库所做的所有修改都会被回滚到没执行该事务之前的状态。

小贴士

这里需要吐槽一下。大家知道，计算机术语基本上全是从英文翻译为中文的。事务的英文是 transaction，直译为“交易”“买卖”的意思。交易就是买的人付钱，卖的人交货，不能付了钱不交货，也不能交了货不付钱。所以交易本身就是一种不可分割的操作。不知道是哪位同学把 transaction 翻译成了“事务”（估计是他也想不出什么更好的词儿，只能使用“事务”）。事务这个词儿完全没有“交易”“买卖”的意思，所以大家理解起来也会比较困难。估计母语是英语的同学可能会更容易理解 transaction 吧。

18.3 MySQL 中事务的语法

事务的本质其实就是一系列数据库操作，只不过这些数据库操作符合 ACID 特性而已。那么，MySQL 是如何将某些操作放到一个事务中去执行的呢？下面就来重点唠叨一下。

18.3.1 开启事务

可以使用下面两种语句来开启一个事务。

- BEGIN [WORK];

BEGIN 语句代表开启一个事务，后边的单词 WORK 可有可无。开启事务后，就可以继续写若干条语句，这些语句都属于刚刚开启的这个事务。

```
mysql> BEGIN;
Query OK, 0 rows affected (0.00 sec)

mysql> 加入事务的语句...
```

- START TRANSACTION;

START TRANSACTION 语句与 BEGIN 语句有相同的功效，都标志着开启一个事务。比如下面这样：

```
mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

mysql> 加入事务的语句...
```

相较于 BEGIN 语句，START TRANSACTION 语句后面可以跟随几个修饰符，如下所示。

- READ ONLY：标识当前事务是一个只读事务，也就是属于该事务的数据库操作只能读取数据，而不能修改数据。

小贴士

其实只读事务只是不允许修改那些其他事务也能访问的表中的数据。对于临时表（使用 CREATE TMEPORARY TABLE 创建的表）来说，由于它们只能在当前会话中可见，所以只读事务可以对临时表进行增删改操作。

- READ WRITE：标识当前事务是一个读写事务，也就是属于该事务的数据库操作既可以读取数据，也可以修改数据。
- WITH CONSISTENT SNAPSHOT：启动一致性读（先不用关心啥是一致性读，后面的章节才会唠叨）。

如果我们想开启一个只读事务，直接把 READ ONLY 修饰符加在 START TRANSACTION 语句后面就好。比如下面这样：

```
START TRANSACTION READ ONLY;
```

如果想在 START TRANSACTION 后面跟随多个修饰符，可以使用逗号将修饰符分开。比如开启一个只读事务和一致性读，可以这样写：

```
START TRANSACTION READ ONLY, WITH CONSISTENT SNAPSHOT;
```

或者开启一个读写事务和一致性读，可以这样写：

```
START TRANSACTION READ WRITE, WITH CONSISTENT SNAPSHOT;
```

不过这里需要注意的一点是，READ ONLY 和 READ WRITE 是用来设置事务访问模式的，就是以只读还是读写的方式来访问数据库中的数据。一个事务的访问模式不能既设置为只读的，也设置为读写的，所以不能同时把 READ ONLY 和 READ WRITE 放到 START TRANSACTION 语句后面。另外，如果不显式指定事务的访问模式，那么该事务的访问模式就是读写模式。

18.3.2 提交事务

开启事务之后就可以继续编写需要放到该事务中的语句了。当最后一条语句写完后就可以提交该事务了。提交的语句也很简单：

```
COMMIT [WORK];
```

COMMIT 语句就代表提交一个事务，后边的 WORK 可有可无。比如前文说的狗哥给猫爷转 10 元钱其实对应 MySQL 中的两条语句。我们就可以把这两条语句放到一个事务中，完整的过程就是下面这样：

```
mysql> BEGIN;
Query OK, 0 rows affected (0.00 sec)

mysql> UPDATE account SET balance = balance - 10 WHERE id = 1;
Query OK, 1 row affected (0.02 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> UPDATE account SET balance = balance + 10 WHERE id = 2;
Query OK, 1 row affected (0.00 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> COMMIT;
Query OK, 0 rows affected (0.00 sec)
```

18.3.3 手动中止事务

如果我们写了几条语句之后发现前面某条语句写错了，可以手动使用下面这个语句将数据库恢复到事务执行之前的样子：

```
ROLLBACK [WORK];
```

ROLLBACK 语句代表回滚一个事务，后边的 WORK 可有可无。比如在写狗哥给猫爷转账 10 元钱所对应的 MySQL 语句时，先给狗哥扣了 10 元，然后一时大意只给猫爷账户上增加了 1 元，此时就可以使用 ROLLBACK 语句进行回滚。完整的过程就是下面这样：

```
mysql> BEGIN;
Query OK, 0 rows affected (0.00 sec)

mysql> UPDATE account SET balance = balance - 10 WHERE id = 1;
Query OK, 1 row affected (0.00 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> UPDATE account SET balance = balance + 1 WHERE id = 2;
Query OK, 1 row affected (0.00 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> ROLLBACK;
Query OK, 0 rows affected (0.00 sec)
```

这里需要强调一下，ROLLBACK 语句是我们程序员在手动回滚事务时使用的。如果事务在执行过程中遇到了某些错误而无法继续执行的话，大部分情况下会回滚失败的语句，在某些情况下会回滚整个事务，比方说在发生了死锁的情况下会回滚整个事务（关于死锁的概念我们会在 22 章唠叨）。

小贴士

这里所说的开启、提交、中止事务的语法只是针对 mysql 客户端程序通过黑框框与服务器进行交互时，用来控制事务的语法。如果大家使用的是其他的客户端程序，比如 JDBC 之类的，则需要参考相应的文档来看看如何控制事务。

18.3.4 支持事务的存储引擎

在 MySQL 中，并不是所有的存储引擎都支持事务的功能，目前只有 InnoDB 和 NDB 存储引擎支持（NDB 存储引擎不是我们的重点）。如果某个事务中包含的语句要修改某个表中的数据，但是该表使用的存储引擎不支持事务，那么对该表所做的修改将无法进行回滚。假设我们有两个表，tbl1 使用支持事务的存储引擎 InnoDB，tbl2 使用不支持事务的存储引擎 MyISAM。它们的建表语句如下所示：

```
CREATE TABLE tbl1 (
    i int
) engine=InnoDB;

CREATE TABLE tbl2 (
```

```
    i int
) ENGINE=MyISAM;
```

我们先开启一个事务，写一条插入语句后再回滚该事务。看看 tbl1 和 tbl2 的表现有什么不同：

```
mysql> SELECT * FROM tbl1;
Empty set (0.00 sec)

mysql> BEGIN;
Query OK, 0 rows affected (0.00 sec)

mysql> INSERT INTO tbl1 VALUES(1);
Query OK, 1 row affected (0.00 sec)

mysql> ROLLBACK;
Query OK, 0 rows affected (0.00 sec)

mysql> SELECT * FROM tbl1;
Empty set (0.00 sec)
```

可以看到，对于使用 InnoDB 存储引擎（支持事务）的 tbl1 来说，我们在插入一条记录再执行 ROLLBACK 语句后，tbl1 恢复到没有插入记录时的状态。再看看 tbl2 表的表现：

```
mysql> SELECT * FROM tbl2;
Empty set (0.00 sec)

mysql> BEGIN;
Query OK, 0 rows affected (0.00 sec)

mysql> INSERT INTO tbl2 VALUES(1);
Query OK, 1 row affected (0.00 sec)

mysql> ROLLBACK;
Query OK, 0 rows affected, 1 warning (0.01 sec)

mysql> SELECT * FROM tbl2;
+------+
| i    |
+------+
|    1 |
+------+
1 row in set (0.00 sec)
```

可以看到，虽然使用了 ROLLBACK 语句来回滚事务，但是插入的那条记录还是留在了 tbl2 表中。

18.3.5 自动提交

MySQL 中有一个系统变量 autocommit，用来自动提交事务。

```
mysql> SHOW VARIABLES LIKE 'autocommit';
+---------------+-------+
| Variable_name | Value |
+---------------+-------+
```

```
| autocommit     | ON     |
+----------------+--------+
1 row in set (0.01 sec)
```

可以看到它的默认值为ON。也就是说在默认情况下，如果不显式地使用START TRANSACTION或者BEGIN语句开启一个事务，那么每一条语句都算是一个独立的事务，这种特性称为事务的自动提交。假如我们在狗哥向猫爷转账10元时不以START TRANSACTION或者BEGIN语句显式开启一个事务，那么下面这两条语句就相当于放到两个独立的事务中执行：

```
UPDATE account SET balance = balance - 10 WHERE id = 1;
UPDATE account SET balance = balance + 10 WHERE id = 2;
```

当然，如果想关闭这种自动提交的功能，可以使用下面两种方法。

- 显式地使用START TRANSACTION或者BEGIN语句开启一个事务。
  这样在本次事务提交或者回滚前会暂时关闭自动提交的功能。
- 把系统变量autocommit的值设置为OFF，就像下面这样：

  ```
  SET autocommit = OFF;
  ```

  这样一来，我们写入的多条语句就算是属于同一个事务了，直到我们显式地写出COMMIT语句把这个事务提交掉；或者显式地写出ROLLBACK语句把这个事务回滚掉。

18.3.6 隐式提交

当使用START TRANSACTION或者BEGIN语句开启了一个事务，或者把系统变量autocommit的值设置为OFF时，事务就不会进行自动提交。如果我们输入了某些语句，且这些语句会导致之前的事务悄悄地提交掉（就像输入了COMMIT语句了一样），那么这种因为某些特殊的语句而导致事务提交的情况称为隐式提交。会导致事务隐式提交的语句有下面这些。

- 定义或修改数据库对象的数据定义语言（Data Definition Language，DDL）。

所谓的数据库对象，指的就是数据库、表、视图、存储过程等这些东西。当使用CREATE、ALTER、DROP等语句修改这些数据库对象时，就会隐式地提交前面语句所属的事务，就像下面这样：

```
BEGIN;

SELECT ... # 事务中的一条语句
UPDATE ... # 事务中的一条语句
... # 事务中的其他语句

CREATE TABLE ... # 此语句会隐式提交前面语句所属的事务
```

- 隐式使用或修改mysql数据库中的表。

在使用ALTER USER、CREATE USER、DROP USER、GRANT、RENAME USER、REVOKE、SET PASSWORD等语句时，也会隐式地提交前面语句所属的事务。

- 事务控制或关于锁定的语句。

当我们在一个事务还没提交或者还没回滚时就又使用START TRANSACTION或者BEGIN

语句开启了另一个事务，此时会隐式地提交上一个事务，就像下面这样：

```
BEGIN;

SELECT ... # 事务中的一条语句
UPDATE ... # 事务中的一条语句
... # 事务中的其他语句

BEGIN; # 此语句会隐式提交前面语句所属的事务
```

在当前的 autocommit 系统变量的值为 OFF，而我们手动把它调为 ON 时，也会隐式地提交前面语句所属的事务。

使用 LOCK TABLES、UNLOCK TABLES 等关于锁定的语句也会隐式地提交前面语句所属的事务。

- 加载数据的语句。

比如使用 LOAD DATA 语句向数据库中批量导入数据时，也会隐式地提交前面语句所属的事务。

- 关于 MySQL 复制的一些语句。

使用 START SLAVE、STOP SLAVE、RESET SLAVE、CHANGE MASTER TO 等语句时也会隐式地提交前面语句所属的事务。

- 其他语句。

使用 ANALYZE TABLE、CACHE INDEX、CHECK TABLE、FLUSH、LOAD INDEX INTO CACHE、OPTIMIZE TABLE、REPAIR TABLE、RESET 等语句时也会隐式地提交前面语句所属的事务。

小贴士

上文提到的这些语句，如果你都认识并且都知道是干嘛用的，那就再好不过了。不认识也不要气馁，这里写出来只是把可能会导致事务隐式提交的情况都列举一下，以保证内容的完整性。至于具体每个语句都是干嘛用的，等遇到了再说。

18.3.7 保存点

如果你已经开启了一个事务，并且已经输入了很多语句，这时忽然发现前面已经执行完成的某个语句的参数写错了，只好使用 ROLLBACK 语句来让数据库状态恢复到事务执行之前的样子，然后一切从头再来。是不是有一种“一夜回到解放前”的感觉？

设计数据库的大叔提出了保存点（savepoint）的概念，就是在事务对应的数据库语句中“打”几个点。我们在调用 ROLLBACK 语句时可以指定回滚到哪个点，而不是回到最初的原点。定义保存点的语法如下：

```
SAVEPOINT 保存点名称;
```

当想回滚到某个保存点时，可以使用下面这个语句（语句中的单词 WORK 和 SAVEPOINT 是可有可无的）：

```
ROLLBACK [WORK] TO [SAVEPOINT] 保存点名称;
```

如果 ROLLBACK 语句后面不跟随保存点名称，则直接回滚到事务执行之前的状态。

如果想删除某个保存点，可以使用这个语句：

```
RELEASE SAVEPOINT 保存点名称;
```

下面还是以狗哥向猫爷转账 10 元的例子来介绍保存点的用法。在执行完扣除狗哥账户的 10 元钱的语句之后，“打”一个保存点：

```
mysql> SELECT * FROM account;
+----+--------+---------+
| id | name   | balance |
+----+--------+---------+
|  1 | 狗哥   |      11 |
|  2 | 猫爷   |       2 |
+----+--------+---------+
2 rows in set (0.00 sec)

mysql> BEGIN;
Query OK, 0 rows affected (0.00 sec)

mysql> UPDATE account SET balance = balance - 10 WHERE id = 1;
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0

# 一个保存点
mysql> SAVEPOINT s1;
Query OK, 0 rows affected (0.00 sec)

mysql> SELECT * FROM account;
+----+--------+---------+
| id | name   | balance |
+----+--------+---------+
|  1 | 狗哥   |       1 |
|  2 | 猫爷   |       2 |
+----+--------+---------+
2 rows in set (0.00 sec)

# 更新语句的参数写错了（应该给账户加10元，语句中加了1元）
mysql> UPDATE account SET balance = balance + 1 WHERE id = 2;
Query OK, 1 row affected (0.00 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> ROLLBACK TO s1;  # 回滚到保存点s1处
Query OK, 0 rows affected (0.00 sec)

mysql> SELECT * FROM account;
+----+--------+---------+
| id | name   | balance |
+----+--------+---------+
|  1 | 狗哥   |       1 |
|  2 | 猫爷   |       2 |
+----+--------+---------+
2 rows in set (0.00 sec)
```

18.4 总结

现实世界的业务场景需要映射到数据库世界。现实世界中的一次状态转换需要满足下面几种特性：

- 原子性；
- 隔离性；
- 一致性；
- 持久性。

需要保证原子性、隔离性、一致性和持久性的一个或多个数据库操作称为事务。

事务在执行过程中有几种状态，分别是：

- 活动的；
- 部分提交的；
- 失败的；
- 中止的；
- 提交的。

关于事务的各种语法实在太多，这里不再一一列举。